



FS



← Découvrez les librairies Python pour la Data Science

 6% de progression

Manipulez le data frame



Nous avons importé notre premier data frame... mais c'est loin d'être le dernier. Une fois nos données correctement importées, la suite logique va être de les manipuler à notre guise.

Voici un cas concret : nous souhaitons accéder à la liste de tous les e-mails des clients ayant contracté un prêt chez nous, à partir du data frame `clients` importé lors du chapitre précédent, pour créer une liste de diffusion pour des offres commerciales préférentielles. Comment faire cela avec Pandas ? C'est que je vous propose de voir à présent !

Naviguez dans le data frame

Dans un data frame, chaque colonne est explicitement nommée, rendant la compréhension de cette dernière plus claire, et permettant d'accéder à une colonne spécifique à partir de son nom !

Pour accéder à une colonne d'un data frame, il suffit d'utiliser la syntaxe `nom_dataframe[nom_colonne]` .

Voilà donc comment accéder à la liste des e-mails à partir du data frame `data` importé lors du chapitre précédent :

```
1 data['email']
```

Ainsi, on accède à la variable `email` de notre data frame `data`. La syntaxe permet une lecture assez claire de ce à quoi on essaie



On peut même décider de stocker cette variable email (au sens des données) dans une autre variable (au sens informatique, cette fois !) pour les besoins de la liste de diffusion présentée ci-dessus. Ici, je crée une variable `email` dans laquelle je stocke tous les e-mails de ma base clients :

```
1 email = data['email']
```

Et si on souhaite accéder à plusieurs colonnes ?

Il suffit de stocker l'ensemble des noms des colonnes auxquelles vous souhaitez accéder dans une liste. Voilà par exemple comment sélectionner le nom ET l'e-mail de nos clients :

```
1 variables = ['nom', 'email']  
2 data[variables]
```

Même si ce bout de code ne comporte que 2 lignes, prenons le temps de bien l'expliquer :

1. On souhaite accéder au nom et à l'e-mail de nos clients. On crée donc dans un premier temps une liste que l'on nomme `variables`, dans laquelle nous stockons l'ensemble des noms des variables.
2. On utilise la même syntaxe que vu précédemment, pour sélectionner dans notre data frame la liste des variables définie.

Et vous verrez que l'ensemble des noms et e-mails de nos clients s'afficheront correctement à l'écran.

Naturellement, il est possible de réaliser ces 2 étapes en une fois :

```
data[['nom', 'email']]
```

Si vous souhaitez utiliser cette écriture, veuillez à bien définir la

liste à l'intérieur de vos crochets de sélection. Cette syntaxe à doubles crochets n'est pas très esthétique et peut être déstabilisante, mais vous évitera de commettre une erreur courante lors de votre apprentissage de Pandas.



nous l'avons spécifié (ici : le nom avec l'e-mail) et non comme elles sont disposées initialement (l'e-mail est avant le nom dans notre data frame `data`). Et si vous vérifiez le data frame initial, vous verrez que l'ordre initial est bien conservé. Ainsi, le processus de sélection nous permet de réorganiser l'ordre des colonnes comme on le souhaite, sans modifier le data frame initial.

Découvrez l'objet Series de Pandas

Avant d'aller plus loin, j'aimerais vous poser une simple question : à votre avis, quels objets manipulons-nous depuis le début du chapitre ? Vous commencez à bien connaître l'objet **data frame**, mais une colonne d'un data frame... n'est plus un data frame ! En effet, il est temps de vous présenter le second objet de Pandas que vous allez énormément utiliser, et qui est intimement lié au data frame : la **Series**.

En effet, chaque colonne de votre data frame est de type **Series** . Vous pouvez vérifier cela par vous-même via la fonction **type** :

```
type( data ['email'])  
pandas.core.series.Series
```

Résultat de la fonction type

Il est important de différencier les deux objets. Même s'ils partagent de nombreuses méthodes, certaines sont néanmoins exclusives à l'un ou l'autre, et cela peut être une source d'erreur lors de l'implémentation d'analyse de données avec Python.

Une différence évidente, mais que je me permets tout de même de noter : un data frame a 2 dimensions avec plusieurs colonnes, alors que la Series n'a qu'une seule dimension. Cela peut fortement vous aider lorsque vous utiliserez certaines

méthodes, car vous pourrez savoir à quel type d'objet vous faites face, en vous basant sur l'affichage que vous avez de ce dernier.

Regardons l'affichage suivant :



	nom	email
0	Laurent Dagenais	LaurentDagenais@rhyta.com
1	Guy Marois	GuyMarois@fleckens.hu
2	Beaufort Lesage	BeaufortLesage@einrot.com
3	Russell Durand	RussellDurand@armyspy.com
4	Alexis Riel	AlexisRiel@rhyta.com
...	...	...
145	Jeromedel Launay	JeromedelLaunay@fleckens.hu
146	Lowell Patel	LowellPatel@rhyta.com
147	Garland Marcheterre	GarlandMarcheterre@fleckens.hu
148	Olivier Aube	OlivierAube@armyspy.com
149	Frontino Brian	FrontinoBrian@einrot.com

150 rows x 2 columns

Exemple de data frame

L'affichage est relativement "joli", avec une organisation en tableau, où chaque colonne est explicitement nommée : c'est un **data frame**. De plus, les informations affichées en bas sont le nombre de lignes et de colonnes. C'est finalement assez logique, car nous avons sélectionné ici **2 colonnes**. Etant donné qu'il n'y a au moins deux colonnes, ça ne peut pas être une Series.

Regardons à présent :

```
data['email']
0      LaurentDagenais@rhyta.com
1      GuyMarois@fleckens.hu
2      BeaufortLesage@einrot.com
3      RussellDurand@armyspy.com
4      AlexisRiel@rhyta.com
...
145     JeromedelLaunay@fleckens.hu
146     LowellPatel@rhyta.com
147     GarlandMarcheterre@fleckens.hu
148     OlivierAube@armyspy.com
149     FrontinoBrian@einrot.com
Name: email, Length: 150, dtype: object
```

Exemple de Series

Vous noterez que l'affichage est un peu plus austère et surtout, que les informations affichées en bas ne sont pas les mêmes : c'est une **Series**.

En information, vous avez notamment le nom de la Series (qui correspond au nom affiché en haut de la colonne lorsque cette dernière fait partie d'un data frame), la longueur (ou le nombre de lignes), et enfin le type de la colonne (entier, chaîne de caractères, etc.).



Une Series ne peut contenir qu'un seul type, alors qu'un data frame, qui est finalement une collection de Series, peut contenir des colonnes de types différents : une colonne d'entiers, une colonne de nombres décimaux, etc.

Il est important de garder ces notions en tête lors de vos futures analyses de données. De nombreuses méthodes sont communes à ces deux objets, alors que d'autres sont spécifiques. Par exemple, l'ensemble des attributs vus au chapitre précédent (`shape` , `head` , `dtypes`) existent pour des Series. Mais certaines méthodes, notamment certaines que nous verrons un peu plus tard dans ce cours, sont exclusives aux data frames, à cause de leur aspect multidimensionnel.

Vous pouvez accéder à l'ensemble des méthodes et attributs des Series directement via [**la documentation officielle de Pandas**](#).

À présent, voyons quelques manipulations possibles avec des Series.

Manipulez les colonnes

Dans cette section, je vous propose de voir quelques manipulations indispensables sur les data frames :

- créer ou supprimer une colonne ;
- renommer une colonne ;
- changer le type d'une colonne ;
- trier un data frame selon une ou plusieurs colonnes.

Modifiez une colonne existante

Avant de voir comment créer une colonne, voyons comment en modifier une déjà existante.

Reprenons la syntaxe vue précédemment,

`nom_dataframe[nom_colonne]` . Cette dernière permet d'accéder à une colonne d'un data frame sans modifier ce dernier. Ce que vous ne savez pas encore, c'est qu'elle permet également de modifier un



Considérons une variable informatique `a` déjà existante :

- on peut y accéder en l'appelant, lui appliquer plusieurs fonctions différentes (comme `print(a)` pour l'afficher) sans modifier sa valeur ;
- mais on peut également définir ou changer sa valeur (`a=4` qui stockera l'entier 4 dans notre variable `a`).

C'est exactement la même chose avec une colonne d'un data frame :

- on peut y accéder et appliquer plusieurs fonctions et/ou méthodes : c'est ce que nous avons fait jusque-là ;
- on peut également changer les valeurs qui se trouvent à l'intérieur.

Par exemple, si j'exécute :

```
1 data['nom'] = 1
```

Cela aura deux effets : cela va modifier la variable `nom` existante en remplaçant toutes les valeurs par 1, et cela transforme également le type de la variable, comme vous pourrez le constater si vous en regardez le contenu :

```
clients['nom']
```

```
0      1
1      1
2      1
```



```
..
145    1
146    1
147    1
148    1
149    1
Name: nom, Length: 150, dtype: int64
```

Transformation d'une colonne du type object vers le type integer

`nom` était de type `object` jusque-là ; elle est à présent de type `integer` car 1 est un entier.

Ainsi, si on le remplace par une valeur fixe (comme c'est le cas ici), cela aura pour effet de remplacer l'ensemble des valeurs de notre colonne par cette valeur fixe, et changera le type de la colonne dans le type de la valeur fixe spécifiée, qu'elle soit numérique ou non.

On peut également remplacer une colonne par un objet de même dimension. Comprenez par là une liste, un array ou une series comprenant l'exact même nombre d'éléments. Par exemple, si je souhaite modifier la colonne `identifiant` par elle-même multipliée par 100, je peux le faire de la façon suivante :

```
1 data['identifiant'] = data['identifiant']*100
```

Je peux aussi décider de la remplacer par des valeurs aléatoires entre 1 et 1 000 :

```
1 data['identifiant'] = np.random.randint(1, 1000, clients.shape[0])
```

Vous noterez que l'array généré pour remplacer les valeurs a strictement la même dimension que la colonne qu'il remplace. Vous aurez une erreur si ce n'est pas le cas, donc soyez vigilant sur cet aspect !

OK je comprends l'idée ! Maintenant j'aimerais revenir à mes anciennes valeurs, ma colonne `nom` ne ressemblant plus à grand-chose...



Il est donc impossible de retrouver nos anciennes valeurs en l'état. Pas de panique cependant si vous avez exécuté ces lignes de votre côté, il vous suffira de recharger les données via la commande `pd.read_csv` pour retrouver les valeurs originales.

En pratique, on ne modifiera une colonne que lorsqu'on est vraiment sûr de nous, afin justement de ne pas supprimer par mégarde des données dont on aurait besoin.

Créez et supprimez une colonne

Maintenant, comment créer une colonne ? De la même façon qu'on en modifie une. On appelle la colonne, comme si elle existait déjà, et on lui attribue une valeur. Ce n'est pas clair ? Laissez-moi vous montrer un exemple :

```
1 data['id'] = data['identifiant'] + 1000
2 data.head()
```

Si vous exécutez ces lignes de votre côté, vous verrez que même si la colonne `id` n'existe pas dans notre data frame, Pandas va la créer et lui attribuer la valeur définie (ici, la valeur se trouvant dans *identifiant*, plus 1 000).

Pour résumer, que ce soit pour modifier ou créer une colonne `col`, la syntaxe sera : `mon_dataframe['col'] = x` où x représente soit une valeur fixe, soit un objet de même dimension que la colonne qu'on souhaite modifier/créer.

Et pour supprimer une colonne existante ?

Il existe officiellement 3 façons. D'abord la méthode `.drop` des data frames :


```
1 data.drop(columns='id')
```

Contrairement aux deux options suivantes, la méthode `.drop` ne modifie pas le data frame existant. Elle renvoie juste une sorte de



supprimer la colonne id. Vous aurez besoin de remplacer votre data frame pour pallier cela :

```
data = data.drop(columns='id') .
```

Ensuite, on peut aussi effectuer cela via le mot clé `del` :

```
1 del data['id']  
2 data.head()
```

Ou finalement via la méthode `.pop` :

```
1 data.pop('id')  
2 data.head()
```

Ces 3 méthodes sont équivalentes : c'est selon votre préférence !

Renommez une colonne

La méthode pour renommer une colonne est `.rename`. On peut ainsi renommer une ou plusieurs colonnes via la syntaxe :

```
mon_dataframe.rename(columns={'ancien nom': 'nouveau  
nom'})
```

Voilà par exemple comment renommer la colonne `identifiant` en `ide` :

```
1 data.rename(columns={'identifiant': 'ide'})
```

On peut naturellement renommer plusieurs colonnes en une fois. Par exemple en modifiant `email` en `mail` :

```
1 data.rename(columns={'identifiant': 'ide', 'email': 'mail'})
```

À noter que la méthode `rename` ne modifie pas le data frame existant. Il existe cependant un argument pour cette méthode (et pour toutes les méthodes similaires) nommé `inplace`, qu'il suffit de fixer à Vrai (`True`) pour pallier cela. Ainsi,



```
inplace=True,
```

est strictement équivalent à

```
data = data.rename(columns={'identifiant':  
                           'ide'})
```

.

Changez le type d'une colonne

La méthode `.astype` permet de changer le type d'une colonne. Par exemple, si on souhaite transformer la colonne `identifiant`, initialement composée d'entiers, en nombres décimaux, on peut le faire de la façon suivante :

```
1 data['identifiant'].astype(float)
```

Il suffira de préciser le type entre parenthèses !

Triez un data frame

En analyse de données, on a régulièrement besoin de trier des données selon une ou plusieurs colonnes. Pandas met à disposition la méthode `.sort_values` pour faire cela très aisément. Il suffit de préciser entre parenthèses la ou les colonnes selon lesquelles il faut trier. Voici quelques exemples :

```
1 # trier selon l'identifiant, par ordre croissant :  
2 data.sort_values('identifiant')
```

```
1 #trier selon l'identifiant par ordre décroissant :  
2 data.sort_values('identifiant', ascending = False)
```



```
1 # trier selon le genre puis le nom, par ordre croissant :  
2 data.sort_values(['genre', 'nom'])
```

Le mot clé **ascending** permet de définir si on souhaite trier par ordre croissant ou décroissant.

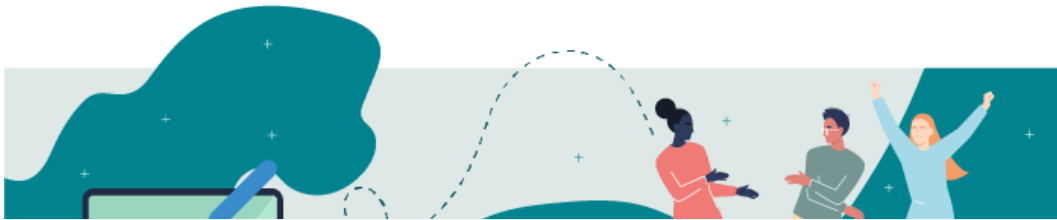
Comment procéder si on souhaite avoir l'un par ordre croissant et l'autre par ordre décroissant ?

Il faudra dans ce cas donner en paramètre à **ascending** une liste de booléens. Voici par exemple comment trier par genre en ordre croissant et par nom en ordre décroissant :

```
1 clients.sort_values(['genre', 'nom'], ascending=[True, False])
```

Maintenant que nous sommes au point avec ces outils, que diriez-vous de mettre tout cela en pratique ?

À vous de jouer



Contexte

Nous allons travailler à présent sur notre fichier de prêts immobiliers.

Pour expliquer rapidement ce fichier, chaque ligne correspond à un prêt qui a été accordé à un de nos clients. Chaque client est identifié par... son identifiant ! Nous avons les informations suivantes :

- la **ville** et le **code postal** de l'agence où le client a contracté le prêt ;
- le **revenu** mensuel du client ;
- les **mensualités** remboursées par le client ;
- la **durée** du prêt contracté, en **nombre de mois** ;
- le **type** de prêt ;
- et enfin le **taux** d'intérêt.

Consignes

Votre rôle cette fois-ci va être de modifier ce jeu de données pour calculer différentes variables nécessaires pour identifier les clients à la limite de leur capacité de remboursement, et déterminer les bénéfices réalisés par la banque.

Voici un exercice pour vous [entraîner à manipuler un data frame](#).

Vérifiez votre travail

Que diriez-vous à présent de comparer vos réponses avec [la correction de l'exercice](#) ?

En résumé

- On peut sélectionner une ou plusieurs colonnes d'un data frame via la syntaxe `mon_dataframe[col]`, où `col` est soit le nom de la colonne, soit un indice.



- Une colonne d'un data frame est une Series Pandas.
- De nombreuses manipulations sont possibles avec/via des Series ; on peut notamment :
 - modifier, ajouter ou supprimer une colonne ;
 - modifier le nom d'une colonne via la méthode `.rename()` ;
 - changer le type d'une colonne via la méthode `.astype()` ;
 - trier un data frame via la méthode `.sort_values()` .

Je vous propose à présent de voir comment filtrer les lignes pour avoir un contrôle total sur notre data frame.



[Avez-vous une suggestion pour nous ?](#)

Chapitre suivant →